

Sidetick — Complete Project Instructions

Trading Education Platform for Indian Retail Traders Full-stack web application: Course Sales Page + Gated Member Portal + Freebies Lead Magnet

Table of Contents

1. [Project Overview](#)
 2. [Brand & Design System](#)
 3. [Technical Stack](#)
 4. [Information Architecture](#)
 5. [Page 1 — Course Landing Page \(Mini Quant\)](#)
 6. [Page 2 — Gated Member Portal](#)
 7. [Page 3 — Freebies Page](#)
 8. [Authentication System](#)
 9. [Payment Integration — Razorpay](#)
 10. [Video Streaming & Piracy Protection](#)
 11. [Database Schema](#)
 12. [Backend API Reference](#)
 13. [Security Requirements](#)
 14. [Admin Dashboard](#)
 15. [Infrastructure & Deployment](#)
 16. [Analytics & Tracking](#)
 17. [SEO Strategy](#)
 18. [Development Timeline](#)
 19. [Environment Variables](#)
 20. [Folder Structure](#)
-

1. Project Overview

What is Sidetick?

Sidetick is a **premium algorithmic trading education platform** targeting Indian retail traders. It teaches TradingView-to-broker automation using Pine Script and the Dhan API. The flagship product is **Mini Quant** — a 22-hour, 11-module course bundled with automation scripts.

Business Context

Metric	Value
Instagram Followers	~40,000
WhatsApp + Telegram Warm Audience	~5,000
Primary Market	Indian retail traders
Flagship Product	Mini Quant (22 hrs, 11 modules)
Payment Gateway	Razorpay (India-first)
Primary Goal	Convert warm social traffic into paying students

Core Objectives

- Convert Instagram/WhatsApp/Telegram warm leads into paying students
- Deliver piracy-protected premium course content post-purchase
- Automate user onboarding immediately after Razorpay payment
- Build a scalable foundation for a QuantInsti-style ecosystem
- Generate lead magnet downloads to grow the WhatsApp funnel

Inspiration Reference

The platform should mirror the professionalism and UX quality of QuantInsti — structured learning paths, progress tracking, trust-building design — but designed specifically for Indian retail traders discovering trading automation for the first time.

2. Brand & Design System

2.1 Color Palette

The Sidetick design uses a **deep navy blue + electric teal green** theme — conveying trust, precision, and modern financial technology.

```

:root {
  /* Primary Brand Colors */
  --color-navy:          #0A1628;    /* Deep navy – primary backgrounds */
  --color-navy-mid:      #0F2040;    /* Slightly lighter navy – cards, sections */
  --color-navy-light:    #162845;    /* Hover states, borders */

  /* Accent – Electric Teal */
  --color-teal:          #00C896;    /* Primary CTA, highlights, links */
  --color-teal-light:    #00E6AB;    /* Hover state of teal */
  --color-teal-dark:     #009E78;    /* Active/pressed state */
  --color-teal-glow:     rgba(0, 200, 150, 0.15); /* Glow effects, card borders */

  /* Supporting Blue */
  --color-blue:          #1A6BFF;    /* Secondary accent, badges */
  --color-blue-light:    #3D85FF;    /* Hover state */
  --color-blue-muted:    rgba(26, 107, 255, 0.12);

  /* Neutral Scale */
  --color-white:         #FFFFFF;
  --color-offwhite:      #F4F7FF;    /* Light page backgrounds (freebies page) */
  --color-text-primary:  #E8EEFF;    /* Primary text on dark bg */
  --color-text-muted:    #8A9BC4;    /* Secondary text, labels */
  --color-text-dark:     #1A2340;    /* Text on light backgrounds */

  /* Status Colors */
  --color-success:       #00C896;    /* Success states (same as teal) */
  --color-warning:       #FFB800;    /* Warnings */
  --color-error:         #FF4B4B;    /* Errors */

  /* Gradients */
  --gradient-hero:       linear-gradient(135deg, #0A1628 0%, #0F2040 50%, #0A1628 100%);
  --gradient-teal:       linear-gradient(135deg, #00C896, #1A6BFF);
  --gradient-card:       linear-gradient(145deg, #0F2040, #162845);
  --gradient-cta:        linear-gradient(135deg, #00C896 0%, #009E78 100%);
}

```

2.2 Typography

CSS

```
/* Fonts – Import via Google Fonts */
/* Display: Syne (geometric, modern, financial feel) */
/* Body: DM Sans (clean, readable, Indian-market friendly) */
/* Mono: JetBrains Mono (code snippets, Pine Script examples) */

@import url('https://fonts.googleapis.com/css2?family=Syne:wght@600;700;800&family=DM+Sans:style=normal;wght@400;500&family=JetBrains+Mono:wght@400;500&display=swap');

:root {
  --font-display: 'Syne', sans-serif;      /* Headlines, hero text */
  --font-body:    'DM Sans', sans-serif;    /* Body copy, UI labels */
  --font-mono:    'JetBrains Mono', monospace; /* Code samples */

  /* Type Scale */
  --text-hero:    clamp(2.8rem, 6vw, 5rem);
  --text-h1:      clamp(2rem, 4vw, 3.25rem);
  --text-h2:      clamp(1.5rem, 3vw, 2.25rem);
  --text-h3:      clamp(1.2rem, 2vw, 1.5rem);
  --text-body:    1rem;      /* 16px */
  --text-sm:      0.875rem;  /* 14px */
  --text-xs:      0.75rem;   /* 12px */
}
```

2.3 Spacing & Layout

CSS

```
:root {  
  --spacing-xs:    0.5rem;    /* 8px */  
  --spacing-sm:    1rem;      /* 16px */  
  --spacing-md:    1.5rem;    /* 24px */  
  --spacing-lg:    2.5rem;    /* 40px */  
  --spacing-xl:    4rem;      /* 64px */  
  --spacing-2xl:   6rem;      /* 96px */  
  
  --radius-sm:     6px;  
  --radius-md:     12px;  
  --radius-lg:     20px;  
  --radius-pill:   999px;  
  
  --max-width:     1200px;  
  --content-width: 780px;  
  
  --shadow-card:   0 4px 24px rgba(0,0,0,0.3), 0 0 0 1px rgba(0,200,150,0.08);  
  --shadow-glow:   0 0 40px rgba(0,200,150,0.2);  
}
```

2.4 Component Patterns

Primary CTA Button

CSS

```
.btn-primary {  
  background: var(--gradient-cta);  
  color: #0A1628;  
  font-family: var(--font-display);  
  font-weight: 700;  
  font-size: 1rem;  
  padding: 0.875rem 2rem;  
  border-radius: var(--radius-pill);  
  border: none;  
  cursor: pointer;  
  transition: transform 0.2s, box-shadow 0.2s;  
}  
  
.btn-primary:hover {  
  transform: translateY(-2px);  
  box-shadow: var(--shadow-glow);  
}
```

Glass Card

CSS

```
.card {  
  background: var(--gradient-card);  
  border: 1px solid rgba(0,200,150,0.12);  
  border-radius: var(--radius-lg);  
  padding: var(--spacing-lg);  
  box-shadow: var(--shadow-card);  
  backdrop-filter: blur(8px);  
}
```

2.5 Design Mood

- **Primary theme:** Dark navy backgrounds with electric teal accents — feels like a Bloomberg terminal meets a modern SaaS product
 - **Freebies page:** Use light `--color-offwhite` background (this page must feel welcoming and accessible, not intimidating)
 - **Animations:** Subtle entry animations (fade + slide-up), teal glow on CTAs, chart-ticker style number counters for social proof
 - **Imagery:** Abstract trading chart lines, waveform overlays, grid/dot patterns as section backgrounds
 - **Icons:** Lucide icons or Phosphor Icons (stroke style, not filled)
-

3. Technical Stack

3.1 Recommended Stack

Layer	Technology	Reason
Frontend Framework	Next.js 15 (App Router)	SSR/SSG, SEO, performance
Styling	TailwindCSS v3	Rapid UI development
UI Components	shadcn/ui	Accessible, customizable
Animations	Framer Motion	Smooth, production-grade
Backend Runtime	Node.js + Express (or Next.js API routes)	Flexible, fast
ORM	Prisma	Type-safe DB queries
Database	PostgreSQL (via Neon or Supabase)	Reliable relational DB
Authentication	Clerk (recommended) or Supabase Auth	OTP support, Indian phone numbers
Video Streaming	Bunny Stream	Tokenized HLS, DRM, India CDN PoPs
Payments	Razorpay	India-native, webhook support
File Storage	Cloudflare R2 or AWS S3	Freebie assets, brand files
Frontend Hosting	Vercel	Zero-config Next.js deployment
Backend Hosting	Railway or Render	Simple containers, auto-deploy
CDN / DDoS	Cloudflare (Free tier)	Speed + protection
Analytics	PostHog + Google Analytics 4	Event tracking + funnel analysis
Error Tracking	Sentry	Production error monitoring
Email / OTP	Twilio (SMS) + Resend (email)	Reliable delivery in India

3.2 Why This Stack vs Alternatives

Alternative	Why NOT Recommended
WordPress + MemberPress	Slow, vulnerable to piracy, plugin hell
Webflow + Memberstack	No backend control for webhooks, limited video security
Kajabi / Teachable	Poor piracy protection, high recurring cost, no India-native payment UX
Custom Django/Rails	Slower to build MVP, smaller talent pool

4. Information Architecture

```

sidetick.in/
├── /                                → Course Landing Page (Mini Quant sales page)
├── /freebies                       → Freebies Lead Magnet Page (public)
├── /login                          → OTP Login Page
├── /dashboard                      → Member Portal (protected)
│   ├── /dashboard/course/[slug]    → Course overview
│   ├── /dashboard/course/[slug]/[lesson] → Video lesson player
│   └── /dashboard/account          → Account settings
├── /api/
│   ├── /api/auth/                 → Auth endpoints
│   ├── /api/payment/              → Razorpay webhook receiver
│   ├── /api/video/                → Signed URL generation
│   ├── /api/freebies/             → Freebie lead capture
│   └── /api/admin/                → Admin-only endpoints
└── /admin                        → Admin dashboard (protected, role=ADMIN)

```

5. Page 1 — Course Landing Page (Mini Quant)

5.1 Page Goal

Convert warm Instagram/WhatsApp leads into paying students for Mini Quant. The page should build trust quickly and drive to Razorpay checkout.

5.2 Page Sections (in order)

Section 1: Sticky Navigation Bar

[Sidetick Logo] [About] [Curriculum] [Testimonials] [FAQs] [Enroll Now →]

- Transparent on scroll-top, solid navy on scroll
- "Enroll Now" button in teal — always visible
- Mobile: hamburger menu

Section 2: Hero Section

Headline: "Automate Your Trading. Stop Guessing. Start Earning."
Sub-headline: "Learn Pine Script, TradingView Alerts & Dhan API Integration in 22 hours – with real scripts you can deploy from Day 1."
CTA: [🔥 Enroll in Mini Quant – ₹___]
Trust nudge: "🔒 Lifetime Access | ✅ 40K+ Community | ⭐ 4.9/5 Rating"
Visual: Animated trading chart / code snippet visual on right

Section 3: Problem Statement

- "Are you stuck in this loop?" — 3 pain points in cards:
 1. Manually watching charts for hours
 2. Strategies look good on paper but fail in execution
 3. Paying for alerts you don't understand

Section 4: Social Proof Bar

- Logos/thumbnails from Instagram posts, YouTube screenshots
- "40,000+ followers | 5,000+ community members | Featured in..."
- Animated counter for student count

Section 5: What You'll Learn (Curriculum)

- 11 module accordion — each expands to list lessons
- Module list:
 1. Introduction to Systematic Trading
 2. Pine Script Fundamentals
 3. Strategy Coding in Pine Script
 4. TradingView Alerts Setup
 5. Dhan API Overview & Authentication
 6. Connecting Alerts to Dhan API
 7. Webhook Automation Architecture
 8. Risk Management Automation
 9. Backtesting & Strategy Validation
 10. Deployment & Live Trading Setup
 11. Scripts Library & Templates
- Show total: "22 hours of content · 11 modules · 60+ lessons · Lifetime access"

Section 6: Mini Quant Feature Highlights

4 feature cards:

Icon	Title	Description
	Ready-to-Deploy Scripts	Get Pine Script templates + webhook handlers usable from Day 1
	Dhan API Mastery	Full coverage of Dhan's order API — market, limit, SL orders
	Mobile Responsive	Watch lessons on phone, tablet, or desktop
	Lifetime Updates	Course updated as TradingView and Dhan API evolve

Section 7: Instructor / Brand Trust Section

- Sidetick founder/instructor photo and short bio
- Instagram follower count + community size
- "As seen on" / featured content screenshots

Section 8: Testimonials

- 3–6 student testimonials with name, photo, result
- Star rating display
- WhatsApp screenshot style cards for authenticity (Indian users trust WA screenshots)

Section 9: Pricing & CTA

[Card: Mini Quant]

Price: ₹X,XXX (original ₹X,XXX – XX% off)

Includes: 22 hours · 11 modules · Scripts library · Lifetime access · Community access

[🔥 Enroll Now – ₹X,XXX]

Guarantee: "7-day money-back guarantee. No questions asked."

Payment icons: UPI · Cards · Net Banking · EMI available

Section 10: FAQ Accordion

Suggested FAQs:

1. Do I need prior coding experience?
2. How do I access the course after payment?
3. What broker do I need? (Dhan)
4. Is the course updated regularly?
5. Can I pay in EMI?
6. How long do I have access?
7. Is there a refund policy?

Section 11: Footer

[Sidetick Logo + tagline]

Links: Home | Freebies | Login | Privacy Policy | Refund Policy | Contact

Social: Instagram | Telegram | YouTube | WhatsApp Community

© 2025 Sidetick. All rights reserved.

GST: XXXXXXXXXXXX

5.3 Conversion Optimizations

- Sticky "Enroll Now" bar appears on mobile after scrolling past hero
- Exit-intent popup: "Wait — get 10% off" → collects email
- WhatsApp chat widget (bottom-right): Floating WhatsApp button
- Scarcity nudge: "Only X seats available at this price" (can be static text or dynamic)
- UPI as first payment option in Razorpay (higher conversion for Indian users)

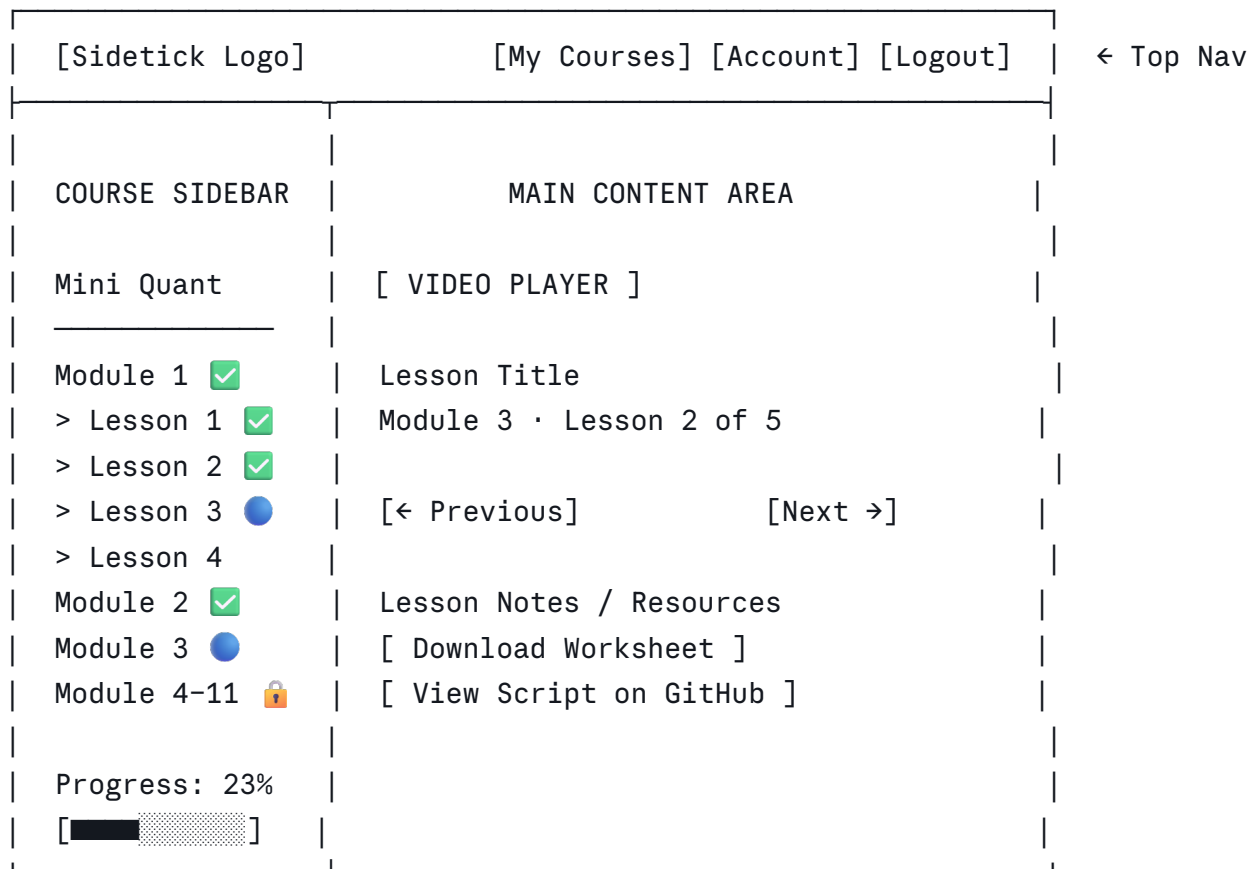
6. Page 2 — Gated Member Portal

6.1 Access Rules

Route Protection Logic:

- User visits /dashboard/*
- Middleware checks: session valid? → YES: check purchase → NO: redirect /login
- Purchase check: user has active purchase for course? → YES: render → NO: redirect /

6.2 Dashboard Layout



6.3 Video Player Requirements

- **Embed Bunny Stream player** — iframe or native player SDK
- Player controls: Play/Pause, Seek, Volume, Playback Speed (0.75x, 1x, 1.25x, 1.5x, 2x), Fullscreen
- **Disable:** Right-click on video, download button, picture-in-picture to external app
- **Watermark overlay:** Student's registered email/phone rendered on video (rotated, semi-transparent) — changes position every 30 seconds
- **Progress save:** On `timeupdate` event, save current position to DB every 30 seconds
- **Resume playback:** On lesson load, seek to last saved timestamp
- **Completion trigger:** Mark lesson complete when >85% watched

6.4 Sidebar — Module/Lesson Navigation

Each Module:

- Module title + lesson count
- Completion status icon:  Complete |  In Progress |  Not Started
- Expandable to show lesson list
- Locked modules shown with  (all unlocked for purchased users)

Each Lesson:

- Lesson title
- Duration (e.g. "14 min")
- Completion checkmark
- Active lesson highlighted in teal

6.5 Progress Tracking Logic

typescript

```
interface LessonProgress {
  userId: string
  lessonId: string
  watchedSeconds: number
  totalSeconds: number
  percentComplete: number // watchedSeconds / totalSeconds * 100
  isCompleted: boolean // percentComplete >= 85
  lastPosition: number // seconds - for resume
  lastWatchedAt: Date
}

// Auto-save every 30 seconds via PATCH /api/progress/:lessonId
// Mark complete when percentComplete >= 85
// Course progress = completedLessons / totalLessons * 100
```

6.6 Account Settings Page (/dashboard/account)

- Display name, email, phone
- Change phone number (re-OTP verification)
- Active purchases list
- Logout button
- No password (passwordless system)

7. Page 3 — Freebies Page

7.1 Page Goal

Capture WhatsApp numbers from non-paying visitors in exchange for free Pine Script templates, strategy guides, and automation resources. Build trust and warm up future buyers.

7.2 Access Flow

```
User visits /freebies
→ Sees freebie cards with blurred/locked download buttons
→ Clicks "Get Free Access"
→ Modal appears: "Enter your WhatsApp number to unlock all freebies"
→ User enters phone → OTP sent → Verified
→ Phone saved to leads table
→ All freebies unlocked for session (localStorage flag)
→ User can download / access all freebies
```

Note: This is a lead magnet, NOT a purchase. Don't require account creation. Simple OTP-verified phone capture is sufficient.

7.3 Page Layout

Header

```
Headline:    "Free Resources for Traders"
Subtext:     "Pine Script templates, strategy guides & automation tools – 100%
free.
            Enter your WhatsApp number to unlock instant access."
CTA:         [{"🔓", "Unlock Free Access"]
```

Freebie Grid (3-column on desktop, 1 on mobile)

Each freebie card contains:

```
[Category Badge: e.g. "Pine Script"]
[Icon / Preview Image]
Title: "EMA Crossover Strategy – Pine Script Template"
Description: 2-3 lines about what it includes
Format badge: PDF | .pine | ZIP | Video
[🔒 Unlock to Download] → after unlock → [↓ Download Now]
```

Suggested Freebies List

#	Title	Type	Description
1	EMA Crossover Pine Script Template	.pine file	Ready-to-use TradingView strategy
2	Dhan API Quickstart Guide	PDF	Step-by-step API setup for beginners
3	Supertrend + RSI Combo Strategy	.pine file	Trend-following automation script
4	TradingView Webhook Setup Checklist	PDF	10-step webhook configuration guide
5	Risk Management Formula Sheet	PDF	Position sizing + SL calculator
6	Top 10 Pine Script Functions Cheatsheet	PDF	Reference guide for Pine coders

Footer CTA

"Want the full 22-hour course with 11 modules and live scripts?"

[🔥 Join Mini Quant ➔]

7.4 Lead Capture Data

```

typescript

interface FreebiesLead {
  id: string
  phone: string           // WhatsApp number with country code
  verifiedAt: Date
  ipAddress: string
  utmSource?: string      // Track where they came from
  utmMedium?: string
  utmCampaign?: string
}
```

8. Authentication System

8.1 Strategy: Passwordless OTP

Recommended: Clerk with phone OTP — handles Indian numbers, session management, and JWT issuance out of the box.

Alternative: Custom OTP using Twilio (SMS) + Redis (OTP store).

8.2 OTP Login Flow

1. User enters phone number (+91XXXXXXXXXX)
2. POST /api/auth/send-otp { phone }
 - Generate 6-digit OTP
 - Store in Redis with 10-minute TTL: key = otp:{phone}, value = {otp, attempts: 0}
 - Send via Twilio SMS (or WhatsApp Business API for WA OTP)
3. User enters OTP
4. POST /api/auth/verify-otp { phone, otp }
 - Check Redis: match + not expired + attempts < 3
 - Increment attempt counter on mismatch
 - On match: issue JWT (httpOnly cookie) + create session record
5. Redirect to /dashboard (if purchase exists) or / (if no purchase)

8.3 Session Management

```
typescript

// JWT Payload
interface JWPayload {
  userId: string
  phone: string
  role: 'STUDENT' | 'ADMIN'
  sessionId: string
  iat: number
  exp: number // 30 days
}

// Session Record (DB)
interface Session {
  id: string
  userId: string
  deviceInfo: string // User-agent
  ipAddress: string
  createdAt: Date
  lastActiveAt: Date
  isRevoked: boolean
}
```

8.4 Route Protection (Next.js Middleware)

typescript

```
// middleware.ts
export function middleware(request: NextRequest) {
  const token = request.cookies.get('sidetick_session')
  const isProtectedRoute = request.nextUrl.pathname.startsWith('/dashboard')
  const isAdminRoute = request.nextUrl.pathname.startsWith('/admin')

  if (isProtectedRoute && !token) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  if (isAdminRoute) {
    // Verify admin role from JWT
    const payload = verifyJWT(token)
    if (!payload || payload.role !== 'ADMIN') {
      return NextResponse.redirect(new URL('/', request.url))
    }
  }
}
```

9. Payment Integration — Razorpay

9.1 Payment Flow (End-to-End)

User clicks "Enroll Now"

↓

Frontend calls POST /api/payment/create-order

↓

Backend creates Razorpay Order (server-side)

Returns { orderId, amount, currency }

↓

Frontend opens Razorpay Checkout modal

User pays via UPI / Card / NetBanking / EMI

↓

Payment success → Razorpay sends webhook to:

POST /api/payment/webhook

↓

Backend validates webhook signature (HMAC-SHA256)

↓

Create/update User record

Grant course access (create Purchase record)

Send welcome email + login link via Resend

↓

Razorpay redirects user to /payment/success?order_id=xxx

↓

Frontend shows success screen → "Check your WhatsApp/Email for login link"

↓

User clicks magic login link → Authenticated → /dashboard

9.2 Razorpay Order Creation

typescript

```
// POST /api/payment/create-order
import Razorpay from 'razorpay'

const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET,
})

export async function createOrder(amount: number, courseId: string, userId?: string) {
  const order = await razorpay.orders.create({
    amount: amount * 100, // Razorpay expects paise
    currency: 'INR',
    receipt: `receipt_${Date.now()}`,
    notes: {
      courseId,
      userId: userId ?? 'new_user',
    },
  })
  return order
}
```

9.3 Webhook Validation

typescript

```
// POST /api/payment/webhook
import crypto from 'crypto'

export async function validateWebhook(
  body: string,
  signature: string
): Promise<boolean> {
  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(body)
    .digest('hex')

  return crypto.timingSafeEqual(
    Buffer.from(expectedSignature),
    Buffer.from(signature)
  )
}

// On valid webhook:
async function handlePaymentSuccess(payload: RazorpayWebhookPayload) {
  const { payment } = payload
  const { courseId, userId } = payment.entity.notes

  // 1. Find or create user
  const user = await upsertUser({ phone: payment.entity.contact })

  // 2. Record purchase
  await db.purchase.create({
    data: {
      userId: user.id,
      courseId,
      razorpayPaymentId: payment.entity.id,
      razorpayOrderId: payment.entity.order_id,
      amount: payment.entity.amount / 100,
      status: 'ACTIVE',
    }
  })

  // 3. Send onboarding email/SMS
  await sendWelcomeEmail(user.email, user.phone)
}
```

9.4 Razorpay Checkout Configuration

javascript

```
const options = {
  key: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID,
  amount: orderAmount,
  currency: 'INR',
  name: 'Sidetick',
  description: 'Mini Quant – Algorithmic Trading Course',
  image: '/logo.png',
  order_id: orderId,
  handler: function(response) {
    // Payment captured – show success UI
    // Actual access provisioning happens via webhook (more reliable)
    window.location.href = `/payment/success?order_id=${response.razorpay_order_id}`
  },
  prefill: {
    name: userName,
    contact: userPhone,
    email: userEmail,
  },
  theme: {
    color: '#00C896', // Teal brand color
  },
  method: {
    upi: true, // UPI first for Indian users
    card: true,
    netbanking: true,
    wallet: false,
    emi: true,
  },
}
const rzp = new window.Razorpay(options)
rzp.open()
```

10. Video Streaming & Piracy Protection

10.1 Platform: Bunny Stream

Bunny Stream is the recommended video hosting platform because it:

- Supports tokenized HLS (signed, expiring playback URLs)
- Has CDN PoPs in India (Mumbai, Chennai) for low latency
- Supports encrypted HLS chunks
- Provides iframe embed with JS SDK
- Costs ~\$0.005/GB storage + ~\$0.01/GB bandwidth (affordable)

10.2 Security Architecture (Multi-Layer)

Layer 1: Signed Playback URLs (Token-based)

- Every playback URL is signed with HMAC-SHA256
- URL contains: videoId + userId + expiresAt
- Expiry: 2 hours from generation
- Token validated by Bunny CDN on every request

Layer 2: HLS Encrypted Chunk Streaming

- Video never served as a single MP4 file
- Delivered as .m3u8 playlist + encrypted .ts chunks
- Encryption keys rotated per session

Layer 3: JWT Session Validation

- API endpoint /api/video/token validates user session before issuing playback URL
- Checks: valid JWT + active purchase + lesson belongs to purchased course

Layer 4: Referrer Validation

- Bunny Stream configured to only allow requests from sidetick.in
- Blocks embeds on other domains

Layer 5: Concurrent Device Limitation

- Max 2 simultaneous streams per user
- Tracked via Redis: key = stream:{userId}, value = [sessionId, ...]
- New login on 3rd device invalidates oldest stream

Layer 6: Dynamic Email/Phone Watermark

- Canvas-based overlay rendered on top of video player
- Shows student's registered phone number (last 4 digits) + email
- Position rotates randomly every 30 seconds
- Semi-transparent (opacity 0.35) – visible in screen recordings

Layer 7: Right-Click & DevTools Protection

- onContextMenu={e => e.preventDefault()} on video container
- CSS: user-select: none on player
- Keyboard shortcut interception for common download shortcuts
- Note: This is soft protection – Layer 1-3 are the real guards

10.3 Signed URL Generation

typescript

```
// GET /api/video/token?lessonId=xxx
import crypto from 'crypto'

export async function generateBunnyToken(
  videoId: string,
  userId: string,
  expiresInSeconds = 7200
): Promise<string> {
  const expiresAt = Math.floor(Date.now() / 1000) + expiresInSeconds
  const securityKey = process.env.BUNNY_SECURITY_TOKEN!
  const path = `/${videoId}/playlist.m3u8`

  const hashBase = securityKey + path + expiresAt + '' // Bunny token format
  const token = crypto.createHash('sha256').update(hashBase).digest('base64')
    .replace(/\+/g, '-').replace(/\//g, '_').replace(\/=/g, '')

  return `https://iframe.mediadelivery.net/play/${process.env.BUNNY_LIBRARY_ID}/${vi
}
```

10.4 Watermark Component

tsx

```
// components/VideoWatermark.tsx
const VideoWatermark = ({ userPhone, userEmail }) => {
  const [position, setPosition] = useState({ top: '20%', left: '10%' })
  const label = `${userEmail} · ${userPhone.slice(-4)}`

  useEffect(() => {
    const interval = setInterval(() => {
      setPosition({
        top: `${Math.random() * 70 + 5}%`,
        left: `${Math.random() * 70 + 5}%`,
      })
    }, 30000)
    return () => clearInterval(interval)
  }, [])

  return (
    <div
      style={{
        position: 'absolute',
        ...position,
        color: 'rgba(255,255,255,0.35)',
        fontSize: '13px',
        fontFamily: 'monospace',
        pointerEvents: 'none',
        userSelect: 'none',
        zIndex: 10,
        transform: 'rotate(-25deg)',
        whiteSpace: 'nowrap',
      }}
    >
      {label}
    </div>
  )
}
```

11. Database Schema

11.1 Prisma Schema

prisma

// schema.prisma

```
generator client {  
  provider = "prisma-client-js"  
}
```

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

// ——— USERS ———

```
model User {  
  id          String    @id @default(cuid())  
  phone       String    @unique  
  email       String?   @unique  
  name        String?  
  role        Role      @default(STUDENT)  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
  
  sessions    Session[]  
  purchases   Purchase[]  
  progress    LessonProgress[]  
}
```

```
enum Role {  
  STUDENT  
  ADMIN  
}
```

// ——— SESSIONS ———

```
model Session {  
  id          String    @id @default(cuid())  
  userId       String  
  user         User      @relation(fields: [userId], references: [id])  
  deviceInfo   String  
  ipAddress    String  
  isRevoked    Boolean   @default(false)  
  createdAt    DateTime  @default(now())  
  lastActiveAt DateTime  @default(now())  
}
```

// ——— COURSES ———

```
model Course {
```

```

model Course {
  id          String    @id @default(cuid())
  slug        String    @unique // e.g. "mini-quant"
  title       String
  description  String
  price       Float
  isPublished Boolean    @default(false)
  createdAt   DateTime   @default(now())

  modules     Module[]
  purchases   Purchase[]
}

// ——— MODULES ———
model Module {
  id          String    @id @default(cuid())
  courseId    String
  course      Course    @relation(fields: [courseId], references: [id])
  title       String
  order       Int
  createdAt   DateTime   @default(now())

  lessons     Lesson[]
}

// ——— LESSONS ———
model Lesson {
  id          String    @id @default(cuid())
  moduleId    String
  module      Module    @relation(fields: [moduleId], references: [id])
  title       String
  order       Int
  bunnyVideoId String    // Bunny Stream video GUID
  durationSecs Int       // Total video duration in seconds
  resources   Json?      // Array of {title, url, type} attachments
  createdAt   DateTime   @default(now())

  progress    LessonProgress[]
}

// ——— PURCHASES ———
model Purchase {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  courseId    String
  course      Course    @relation(fields: [courseId], references: [id])

```

```

    razorpayOrderId      String      @unique
    razorpayPaymentId    String      @unique
    amount               Float
    status               PurchaseStatus @default(ACTIVE)
    purchasedAt          DateTime     @default(now())
}

```

```

enum PurchaseStatus {
    ACTIVE
    REFUNDED
    REVOKED
}

```

```

// — LESSON PROGRESS —————
model LessonProgress {
    id          String      @id @default(cuid())
    userId      String
    user        User        @relation(fields: [userId], references: [id])
    lessonId    String
    lesson      Lesson      @relation(fields: [lessonId], references: [id])
    watchedSeconds Int      @default(0)
    lastPosition Int        @default(0) // Resume point in seconds
    isCompleted Boolean     @default(false)
    lastWatchedAt DateTime  @default(now())

    @@unique([userId, lessonId])
}

```

```

// — FREEBIE LEADS —————
model FreebiesLead {
    id          String      @id @default(cuid())
    phone       String      @unique
    verifiedAt  DateTime     @default(now())
    ipAddress   String
    utmSource   String?
    utmMedium   String?
    utmCampaign String?
}

```

```

// — FREEBIE RESOURCES —————
model FreebieResource {
    id          String      @id @default(cuid())
    title       String
    description  String
    fileType    String      // "PDF" | "PINE" | "ZIP" | "VIDEO"
    downloadUrl String      // Cloudflare R2 signed URL base
}

```

```
category      String      // "Pine Script" | "Guide" | "Checklist"
isPublished   Boolean    @default(true)
order         Int
}
```

12. Backend API Reference

12.1 Auth Endpoints

Method	Endpoint	Description	Auth Required
POST	<code>/api/auth/send-otp</code>	Send OTP to phone	No
POST	<code>/api/auth/verify-otp</code>	Verify OTP, issue session	No
POST	<code>/api/auth/logout</code>	Revoke session	Yes
GET	<code>/api/auth/me</code>	Get current user	Yes

12.2 Payment Endpoints

Method	Endpoint	Description	Auth Required
POST	<code>/api/payment/create-order</code>	Create Razorpay order	No
POST	<code>/api/payment/webhook</code>	Receive Razorpay webhook	No (signature validated)
GET	<code>/api/payment/verify?order_id=</code>	Check payment status	Yes

12.3 Video Endpoints

Method	Endpoint	Description	Auth Required
GET	<code>/api/video/token?lessonId=</code>	Get signed Bunny playback URL	Yes + Purchase Check

12.4 Progress Endpoints

Method	Endpoint	Description	Auth Required
GET	<code>/api/progress/:courseId</code>	Get all progress for a course	Yes
PATCH	<code>/api/progress/:lessonId</code>	Update lesson progress	Yes

12.5 Freebies Endpoints

Method	Endpoint	Description	Auth Required
GET	<code>/api/freebies</code>	List all freebie resources	No
POST	<code>/api/freebies/verify</code>	Capture phone + OTP for freebies	No
GET	<code>/api/freebies/download/:id</code>	Get download URL (after phone verified)	Phone OTP only

12.6 Admin Endpoints

Method	Endpoint	Description	Auth Required
GET	<code>/api/admin/users</code>	List all users	ADMIN
PATCH	<code>/api/admin/users/:id/revoke</code>	Revoke user access	ADMIN
GET	<code>/api/admin/analytics</code>	Revenue + engagement stats	ADMIN
POST	<code>/api/admin/courses</code>	Create/update course	ADMIN
POST	<code>/api/admin/lessons</code>	Create/update lesson	ADMIN

13. Security Requirements

13.1 Mandatory Security Controls

Control	Implementation
HTTPS Everywhere	Enforced at Vercel + Cloudflare level
JWT Authentication	httpOnly cookie, SameSite=Strict
Rate Limiting	5 OTP requests/hour per phone via Redis
SQL Injection Prevention	Prisma ORM (parameterized queries)
Webhook Signature Validation	HMAC-SHA256 on every Razorpay webhook
XSS Prevention	Next.js built-in + Content Security Policy header
CSRF Protection	SameSite=Strict cookie
Concurrent Session Limit	Max 2 active video streams per user
Session Revocation	Admin can invalidate sessions instantly
Secure Headers	Via Next.js <code>headers()</code> + Cloudflare

13.2 Security Headers

typescript

```
// next.config.ts
const securityHeaders = [
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'X-Frame-Options', value: 'DENY' },
  { key: 'X-XSS-Protection', value: '1; mode=block' },
  { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
  { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
  {
    key: 'Content-Security-Policy',
    value: [
      "default-src 'self'",
      "script-src 'self' 'unsafe-inline' https://checkout.razorpay.com",
      "frame-src https://iframe.mediadelivery.net",
      "img-src 'self' data: https:",
      "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
    ].join('; ')
  }
]
```

13.3 Cloudflare Rules

- **Enable DDoS Protection** (Auto)
 - **Bot Fight Mode:** ON
 - **Rate Limiting Rule:** Max 100 req/min per IP on `/api/*`
 - **WAF Rule:** Block requests where Referer is not `sidetick.in` for video endpoints
 - **Cache Rules:** Cache static assets 30 days, never cache `/api/*` or `/dashboard/*`
-

14. Admin Dashboard

14.1 Admin Route: `/admin`

Access restricted to users with `role = ADMIN`.

14.2 Admin Features

Dashboard Overview

- |— Total Revenue (today / this month / all-time)
- |— New Enrollments (chart: last 30 days)
- |— Active Students count
- |— Completion Rate
- └— Top Referral Sources

Users Tab

- |— Search by phone / email
- |— View purchase history
- |— Revoke access
- └— Export CSV

Courses Tab

- |— Create / Edit course
- |— Add / reorder modules
- |— Upload lesson (Bunny Stream) or paste Video ID
- └— Publish / unpublish

Freebies Tab

- |— Add / edit freebie resources
- |— View lead capture count
- └— Export leads CSV

Analytics Tab

- |— Revenue by day / week / month
- |— Lesson completion rates
- |— Drop-off points (which lessons students abandon)
- └— Device breakdown (mobile vs desktop)

Coupons Tab

- |— Create discount code (% or flat ₹)
- |— Set max uses + expiry
- └— Track coupon usage

15. Infrastructure & Deployment

15.1 Services Map

PRODUCTION

[User Browser]

↓

[Cloudflare CDN + WAF + DDoS Protection]

↓

[Vercel] → Next.js App (Frontend + API Routes)

↓

[Railway / Render] → Express Backend (if separate)

↓

[Neon / Supabase] → PostgreSQL Database

[Upstash Redis] → OTP Store + Rate Limiting + Stream Tracking

[Bunny Stream] → Encrypted HLS Video Delivery

[Cloudflare R2] → Freebie File Storage

[Twilio] → SMS OTP

[Resend] → Transactional Email

[PostHog] → Product Analytics

[Sentry] → Error Monitoring

15.2 Deployment Commands

bash

Frontend (Vercel – auto-deploys on git push to main)

vercel --prod

Database migrations

npx prisma migrate deploy

Generate Prisma client

npx prisma generate

15.3 Scalability Phases

Phase	Scope	Timeline
MVP (Phase 1)	Landing page + Member portal + Freebies + Razorpay + Bunny Stream	Weeks 1-4
Growth (Phase 2)	Blog/SEO, webinars, course certificates, quiz system, coupons	Months 2-4
Scale (Phase 3)	AI trading assistant, backtesting engine, mobile app, broker integrations	Months 5-12

16. Analytics & Tracking

16.1 Events to Track (PostHog)

```

typescript

// Key events
posthog.capture('page_viewed', { page: 'landing' })
posthog.capture('cta_clicked', { cta: 'hero_enroll', position: 'hero' })
posthog.capture('checkout_opened', { courseId, amount })
posthog.capture('payment_success', { courseId, amount, method: 'upi' })
posthog.capture('lesson_started', { courseId, moduleId, lessonId })
posthog.capture('lesson_completed', { courseId, lessonId, watchTime })
posthog.capture('freebie_unlocked', { phone: 'hashed' })
posthog.capture('freebie_downloaded', { resourceId })

```

16.2 Meta Pixel Events

```

javascript

fbq('track', 'ViewContent', { content_name: 'Mini Quant Landing' })
fbq('track', 'InitiateCheckout', { value: price, currency: 'INR' })
fbq('track', 'Purchase', { value: price, currency: 'INR' })
fbq('track', 'Lead') // On freebie phone capture

```

16.3 Google Analytics 4

```

javascript

gtag('event', 'begin_checkout', { currency: 'INR', value: price })
gtag('event', 'purchase', { transaction_id: orderId, value: price, currency: 'INR' })

```

17. SEO Strategy

17.1 On-Page SEO

html

```
<!-- Landing Page Meta -->
<title>Mini Quant – Learn TradingView Automation & Pine Script | Sidetick</title>
<meta name="description" content="Master algorithmic trading with Pine Script and Dh
<meta property="og:title" content="Mini Quant – Automate Your Trading Strategy" />
<meta property="og:image" content="https://sidetick.in/og-image.jpg" />

<!-- Structured Data -->
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Course",
  "name": "Mini Quant",
  "provider": { "@type": "Organization", "name": "Sidetick" },
  "description": "Learn TradingView automation, Pine Script, and Dhan API integratio
  "offers": { "@type": "Offer", "price": "XXXX", "priceCurrency": "INR" }
}
</script>
```

17.2 Future Content Strategy (Phase 2+)

Target keywords:

- "pine script tutorial in hindi"
- "dhan api python tutorial"
- "tradingview webhook automation"
- "algo trading for beginners india"
- "how to automate trading strategies"

18. Development Timeline

Phase	Tasks	Duration
Week 1	UI/UX design (Figma/v0), design system setup, DB schema	5 days
Week 2	Frontend: Landing page + Freebies page + Auth UI	5 days
Week 3	Backend: Auth API + Razorpay integration + Webhook handler	5 days
Week 4	Member portal + Video player + Progress tracking + Admin	5 days
Buffer	QA, testing, Bunny Stream setup, Cloudflare config, deploy	3-4 days

Total estimated timeline: 3.5 – 4 weeks

Pre-Development Checklist

- ☐ Razorpay account activated (test + live keys)
- ☐ Bunny Stream account created, library configured
- ☐ Twilio account with Indian SMS approved
- ☐ Resend (or SendGrid) account for emails
- ☐ Domain registered: sidetick.in (or .com)
- ☐ Cloudflare connected to domain
- ☐ Figma access to brand assets (logo, colors, fonts)
- ☐ Course videos ready for upload
- ☐ All written copy for landing page, freebies page

19. Environment Variables

```
bash
```

```
# — App —————
```

```
NEXT_PUBLIC_APP_URL=https://sidetick.in
```

```
NODE_ENV=production
```

```
# — Database —————
```

```
DATABASE_URL=postgresql://user:password@host:5432/sidetick
```

```
# — Redis (Upstash) —————
```

```
UPSTASH_REDIS_REST_URL=
```

```
UPSTASH_REDIS_REST_TOKEN=
```

```
# — JWT —————
```

```
JWT_SECRET=<random 64-char hex string>
```

```
JWT_EXPIRES_IN=30d
```

```
# — Razorpay —————
```

```
RAZORPAY_KEY_ID=
```

```
RAZORPAY_KEY_SECRET=
```

```
RAZORPAY_WEBHOOK_SECRET=
```

```
NEXT_PUBLIC_RAZORPAY_KEY_ID= # Public key for frontend checkout
```

```
# — Bunny Stream —————
```

```
BUNNY_LIBRARY_ID=
```

```
BUNNY_API_KEY=
```

```
BUNNY_SECURITY_TOKEN= # For URL signing
```

```
NEXT_PUBLIC_BUNNY_CDN_URL=https://your-pullzone.b-cdn.net
```

```
# — Twilio (SMS OTP) —————
```

```
TWILIO_ACCOUNT_SID=
```

```
TWILIO_AUTH_TOKEN=
```

```
TWILIO_PHONE_NUMBER=
```

```
# — Resend (Email) —————
```

```
RESEND_API_KEY=
```

```
# — Cloudflare R2 (Freebies Storage) —————
```

```
R2_ACCOUNT_ID=
```

```
R2_ACCESS_KEY_ID=
```

```
R2_SECRET_ACCESS_KEY=
```

```
R2_BUCKET_NAME=sidetick-freebies
```

```
# — Analytics —————
```

```
NEXT_PUBLIC_POSTHOG_KEY=
```

```
NEXT_PUBLIC_GA_MEASUREMENT_ID=
```

```
NEXT_PUBLIC_META_PIXEL_ID=
```

NEXT_PUBLIC_META_PIXEL_ID=

— Sentry —

SENTRY_DSN=

20. Folder Structure

```

sidetick/
├─ app/                                # Next.js App Router
│   ├─ (public)/
│   │   ├─ page.tsx                    # Landing page (/)
│   │   ├─ freebies/
│   │   │   └─ page.tsx                # Freebies page
│   │   └─ login/
│   │       └─ page.tsx                # OTP login
│   ├─ (protected)/
│   │   └─ dashboard/
│   │       ├─ layout.tsx              # Auth guard layout
│   │       ├─ page.tsx                # Course overview
│   │       ├─ course/
│   │       │   └─ [slug]/
│   │       │       └─ page.tsx
│   │       │       └─ [lessonId]/
│   │       │           └─ page.tsx    # Video player
│   │       └─ account/
│   │           └─ page.tsx
│   └─ admin/
│       └─ ...                          # Admin dashboard pages
├─ api/
│   ├─ auth/
│   │   ├─ send-otp/route.ts
│   │   ├─ verify-otp/route.ts
│   │   ├─ logout/route.ts
│   │   └─ me/route.ts
│   ├─ payment/
│   │   ├─ create-order/route.ts
│   │   ├─ webhook/route.ts
│   │   └─ verify/route.ts
│   ├─ video/
│   │   └─ token/route.ts
│   ├─ progress/
│   │   └─ [lessonId]/route.ts
│   ├─ freebies/
│   │   ├─ route.ts
│   │   ├─ verify/route.ts
│   │   └─ download/[id]/route.ts
│   └─ admin/
│       └─ ...
├─ layout.tsx                          # Root layout
└─ globals.css

├─ components/
│   └─ ui/                             # shadcn/ui components

```

```

|   └─ landing/                                # Landing page sections
|   |   └─ Hero.tsx
|   |   └─ Curriculum.tsx
|   |   └─ Testimonials.tsx
|   |   └─ Pricing.tsx
|   |   └─ FAQ.tsx
|   └─ portal/                                # Member portal components
|   |   └─ VideoPlayer.tsx
|   |   └─ VideoWatermark.tsx
|   |   └─ CourseSidebar.tsx
|   |   └─ ProgressBar.tsx
|   └─ freebies/
|   |   └─ FreebieCard.tsx
|   |   └─ PhoneCaptureModal.tsx
|   └─ shared/
|       └─ Navbar.tsx
|       └─ Footer.tsx
|       └─ OTPInput.tsx
|
└─ lib/
    └─ db.ts                                # Prisma client singleton
    └─ redis.ts                            # Upstash Redis client
    └─ auth.ts                            # JWT utilities
    └─ razorpay.ts                        # Razorpay helpers
    └─ bunny.ts                          # Bunny Stream helpers
    └─ email.ts                          # Resend email templates
    └─ sms.ts                            # Twilio OTP helpers
    |
└─ middleware.ts                        # Route protection
└─ prisma/
    └─ schema.prisma
└─ public/
    └─ logo.png
    └─ og-image.jpg
└─ .env.local
└─ .env.example
└─ next.config.ts
└─ tailwind.config.ts
└─ package.json

```

Important Note: This document is the single source of truth for the Sidetick platform build. All developers, designers, and stakeholders should refer to this document before making architectural or design decisions. Any significant deviations should be discussed and reflected back here.